

DataVault Deployment Automation Platform

DataVault Technologies - GitOps Remediation Platform Guide

Kubernetes · ArgoCD · Terraform · GitHub Actions

1. Project Structure

datavault-gitops/

app/

2. End-to-End Flow

The platform follows a linear pipeline from developer push to running pods:

1. **Developer** pushes code to GitHub
2. **GitHub Actions** runs tests, builds Docker image, pushes to ECR, updates `k8s/deployment.yaml`, commits back to Git (audit trail)
3. **Git** remains the single source of truth for both application code and Kubernetes manifests
4. **ArgoCD** detects manifest changes and syncs the k3s cluster
5. **Kubernetes** runs 2+ replicas with probes, HPA, and self-healing
6. **Clients** reach the API via NodePort `http://<EC2-IP>:30080`

Flow Diagram

Developer Push

?

Infrastructure (Terraform): ECR stores images · EC2 t3.small runs k3s + ArgoCD + app workloads

3. Layer by Layer

3.1 Application (`app/`)

The FastAPI app exposes:

Endpoint	Role
`/health`	Liveness - if it fails, Kubernetes restarts the pod
`/ready`	Readiness - if it fails, the Service stops sending traffic
`/api/*`	Business API (requires `x-api-key` from Secret)

The Dockerfile packages main.py on Python 3.11 and runs uvicorn on port 8000.

3.2 Kubernetes Manifests (`k8s/`)

File	Purpose	Connects to
configmap.yaml	Non-sensitive env: `APP_ENV`, `LOG_LEVEL`	Injected into pods via `envFrom`
secret.yaml	`API_KEY` (base64)	Same - app authentication
deployment.yaml	2 replicas, rolling updates, probes, ECR image	Pulls from ECR; reads ConfigMap + Secret
service.yaml	NodePort 30080 -> pod port 8000	Access: `http://<EC2-IP>:30080`
hpa.yaml	Min 2, max 5 pods at 70% CPU	Requires metrics-server (installed in `user_data.sh`)

Self-healing: If a pod dies, the Deployment controller creates a new one. Liveness failures trigger restarts. Readiness keeps unhealthy pods out of traffic during rollouts.

3.3 Terraform (`terraform/`)

Resource	Purpose
ECR (`datavault-api`)	Private store for images built by CI
EC2 t3.small	Runs k3s (Kubernetes) + ArgoCD
IAM role	Lets the node pull from ECR
Security group	SSH (22), API (30080), ArgoCD UI (30081)
S3 backend	Versioned, shared Terraform state

`user\_data.sh` on first boot:

1. Installs **k3s**
2. Installs **metrics-server** (for HPA)
3. Configures **ECR login** (cron every 6 hours)
4. Installs **ArgoCD**

After terraform apply, use outputs (ecr_repository_url, k3s_public_ip) to update k8s/deployment.yaml image URI and configure CI secrets.

3.4 GitHub Actions (`.github/workflows/ci.yml`)

On push to main:

1. **test** - `pytest tests/`
2. **build-and-deploy** - build image from `app/`, push to ECR with tag `github.sha`
3. **Update** `k8s/deployment.yaml` with new image + `GIT\_COMMIT`
4. **Commit & push** - that commit is the **FCA audit record** (who, what, when)

ArgoCD sees the manifest change and rolls out the new version automatically.

3.5 ArgoCD (`.argocd/application.yaml`)

Applied once on the cluster:

```
kubectl apply -f argocd/application.yaml
```

ArgoCD watches your GitHub repo's k8s/ folder. When Git and the cluster differ, it syncs the cluster to match Git.

```
| Old (Valentine's Day) | New (GitOps) |
|-----|-----|
| SSH + manual restart | Git commit -> ArgoCD sync |
| No rollback | `git revert` -> ArgoCD redeploy previous version |
| No audit trail | Git log + ArgoCD sync history |
```

4. How Each Problem Is Solved

```
| Valentine's Day Problem | Solution in This Repo |
|-----|-----|
| Manual SSH deploys | CI + ArgoCD - no SSH for app deploys |
| No rollback | `git revert` + ArgoCD sync |
| No deployment audit trail | Every deploy = Git commit (author, SHA, message) |
| No self-healing | Kubernetes restarts failed pods; liveness on `/health` |
| Config drift between servers | One declarative `k8s/` set for the whole cluster |
| Knowledge in one engineer's head | Everything in Git + Terraform |
```

5. Setup Order

When you deploy for the first time, follow this sequence:

1. Create S3 bucket for Terraform state -> configure `terraform/backend.tf`
2. Run `terraform apply` -> provisions ECR + EC2 with k3s
3. SSH to EC2 -> verify `kubectl get nodes` shows **Ready**
4. Update `k8s/deployment.yaml` image with real AWS account ID from `terraform output`
5. Apply manifests: `kubectl apply -f k8s/` (or wait for ArgoCD after step 6)
6. Register repo in ArgoCD -> `kubectl apply -f argocd/application.yaml`
7. Add GitHub secrets: `AWS\_ACCESS\_KEY\_ID`, `AWS\_SECRET\_ACCESS\_KEY`
8. Push to `main` -> CI runs -> ArgoCD deploys automatically

6. Demo Scenarios

Use these during your Week 2 presentation to Priya Mehta and the Tier 1 bank compliance representative.

Demo	Command / Action
Self-healing	`kubectl delete pod <name>` -> new pod appears within ~30 seconds
Rollback	Deploy broken version -> `git revert <commit>` -> push -> ArgoCD rolls back
Audit trail	`git log k8s/deployment.yaml` - shows who deployed what and when
HPA scaling	Generate CPU load -> `kubectl get hpa` shows replica count increasing
Teardown	`terraform destroy` - removes EC2 and ECR from AWS

Presentation Talking Points

- * **The Git log IS the audit trail** - show commit author, message, timestamp, and PR approval
- * **Contrast with old process:** "This used to take 40 minutes and required SSH into three servers. It now takes minutes and requires a Git push."
- * **Self-healing:** Compare to 2 hours 33 minutes on Valentine's Day - kill a pod live and time the recovery

7. Placeholders You Must Fill In

Before going live, replace these placeholders:

Location	What to change
k8s/deployment.yaml	Replace `000000000000` with your AWS account ID (`terraform output aws_account_id`)
terraform/backend.tf	Your S3 state bucket name
terraform apply	Pass `-var="ssh_key_name=YOUR_KEY"` for EC2 SSH access
argocd/application.yaml	Your real GitHub repository URL
GitHub repo secrets	`AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` for ECR push

8. Architecture Summary

Tool	What It Does	Why DataVault Needs It
------	--------------	------------------------

```
|-----|-----|-----|
| **k3s** | Lightweight Kubernetes on a single EC2 instance | Self-healing, scaling, rollbacks - replaces 3 bare-metal servers |
| **ArgoCD** | GitOps - syncs cluster to Git | Every deployment and rollback is a Git commit (FCA audit trail) |
| **Kubernetes manifests** | Declarative desired state | What is in Git is what runs in production - no config drift |
| **HPA** | Auto-scales pods on CPU | Handles end-of-month compliance report spikes |
| **Probes** | Liveness + readiness health checks | Crashed pods restart; unready pods get no traffic |
| **GitHub Actions** | CI: test, build, push, update manifest | Automates build; triggers the GitOps loop |
| **Terraform** | Provisions ECR + EC2 | Reproducible infrastructure anyone can recreate |
| **Amazon ECR** | Private container registry | Secure image storage; cluster pulls during deploy |
```

Document generated for DataVault Technologies - Amdari Internship Programme

Confidential - Platform Engineering Remediation (90-day plan)